

### The Third Assignment:

#### Group Details:

| No. | Student ID | Student Name             |
|-----|------------|--------------------------|
| 1   | 445000937  | سالم خالد سالم اللحاني   |
| 2   | 445001339  | حاتم محمد حمود الحربي    |
| 3   | 444006177  | باسل احمد ابراهيم الشريف |

#### Project Title (**Maximum 10 words**):

SMART ACADIMIC TASK MANAGER

#### Project Description (**Minimum 100 words**):

The Smart academic task manager is a software that uses artificial intelligence to help students Prioritizing their school assignments tasks and projects By keeping things organized without distractions and complicated steps. It lets you and put stuff like homework assignments or quizzes and maybe bigger projects and exams too, And you can add deadlines and Quick notes about them if you need to, And what we are planning to do with the artificial intelligence That it will have access to all these information so he can make decisions on which category will best fit each task and well guess how long each task would take and decides on the priorities that make sense such as a task with a deadline that's due tomorrow is more important then a task with a due date next week.

#### Define the AI technologies that will be applied in your project? (**Minimum 500 words**):

The Smart Academic Task Manager or SATM basically gathering a bunch of AI stuff to look at student tasks and figure out what to suggest next. For example if student has task related to his field , the system will analyses it in bunch of parts ,how long it talk to complete it , give priority to the right part , estimate the time to apply it .It is not just one big model doing everything but more like separate parts that work together each handling its own job. I think that makes sense for keeping things updated later on without messing up the whole setup.

When students in their tasks like in regular words the system has to make sense of it all. Natural Language Processing is the main thing for that. It starts by breaking down the text getting rid of common words that do not add much and cleaning it up a bit. Then it turns those words into numbers somehow, using methods like TF-IDF or those embedding things. That way the computer can actually work with it. It seems kind of basic but necessary I guess.

Classifying tasks is another part. The idea is to sort them into categories, you know, like homework tests or bigger projects. They use machine learning for this training it on examples that are already labeled. Stuff like logistic regression or support vector machines those come up a lot. Once it is trained it just guesses the type for new ones. This helps because different tasks need different approaches right.

Estimating time feels tricky. The system tries to predict how long something will take based on the type how much text there is difficulty and what the user has done before. It is set up as a regression problem so models like linear regression or decision trees fit. As more info gets added over time it should get better I suppose. But it might not always be spot on at

first.

For deciding order there is this priority scoring. It mixes deadlines time estimates task kinds and how important the user says it is. Starts with some rules to set scores then tweaks them using data from past behavior That hybrid way keeps it understandable while still learning. I am not totally sure how balanced that is but it sounds practical.

Recommendations come in for scheduling too. It suggests when to do tasks fitting them into free spots. A simple algorithm does the basic planning first, to make sure it works. Then maybe fancier things like filtering from other users or learning from tries. Adapting to what the person likes is key I think.

Data gets stored for everything, tasks and predictions and how users react. That feeds back to improving the models now and then. They check with things like accuracy rates or error measurements and even ask about satisfaction. It is all to make the system better over time.

The whole thing is modular so each AI piece talks to others through set ways. Easy to swap one out say upgrade a simple classifier to something deeper if needed. That flexibility stands out.

Overall, we prefer merge NLP because it have the ability to deal with varuios types in one tool to help the students, and it might be grow as you need

Define Dataset that will be used in your project with Samples? (**Minimum 500 words**):

### **Dataset description**

The SATM project's Data set is one that is specifically designed to suit the project's needs but is made to replicate a student's tasks. As no data set was available in the public domain, which exactly meets the criteria of this system a synthetic data set was created to serve multiple machine learning components of this project. The file has a data set of students task entries where each row presents an academic task these tasks include different types of assignments that include homework quizzes exams projects etc. The dataset Was purposely designed to be semi structured And a bit noisy to simulate the way which real people input data with inconsistent spelling and unnecessary symbols.

### **Characteristics of the Dataset**

The data set contains the following  
the user enters a written description of the tasks and the description may contain informal language abbreviation and other formatting symbols numbers all kinds of weird patterns And tasks and for example(homework, quiz, project ,exam) and we use these tasks to classify them as labels

the difficulty level of the task range from(easy, medium, hard) and the user can specify the days remaining until the deadline of the tasks and this is one of the things that can help the model to decide how urgent the task is

### Purpose of the dataset

The SATM trust him serves different functions with the help of the data set One of them is classifying the tasks this data set trains a supervised machine learning model that will classify a task into categories based on the task description and the model will have a time prediction feature for each task regression model using task type, task difficulty, and task Length as the characteristics that we will use to train the model to predict the time to be taken for each task completion.

### Ranking system

And we will make using the data set a hybrid priority system by adopting rule based logic and data based logic. And and we will use natural language processing(NLP) and the text field to process The field using NLP techniques like tokenization, stop word removal, and vectorization to make the model understand the data set.

Tha dataset Contains around 1000 entries And it's not clean and there are different types of noise in the data set such as blending of capital and the use of informal abbreviation like(hw for homework) and Special characters and symbol and labeling has a few errors And because of that the data set needs to be cleaned and prepared to be used in the machine learning phase

### Dataset sample

|    | A                           | B        | C          | D               | E                  | F          |
|----|-----------------------------|----------|------------|-----------------|--------------------|------------|
| 1  | Task Text                   | Category | Difficulty | Deadline (days) | Time Taken (hours) | Importance |
| 2  | finish coding assignment    | Exam     | hard       | 4               | 2.6                | low        |
| 3  | write report draft          | Homewor  | Medium     | 5               | 4.9                | low        |
| 4  | math problems 1-8           | Project  | Hard       | 7               | 2.8                | high       |
| 5  | project research            | Exam     | easy       | 7               | 2.4                | Medium     |
| 6  | quiz 1                      | Quiz     | Hard       | 5               | 2                  | low        |
| 7  | study notes                 | Exam     | easy       | 3               | 5.1                | high       |
| 8  | finish coding assignment    | Homewor  | easy       | 2               | 2.3                | high       |
| 9  | project research            | Quiz     | easy       | 7               | 4.6                | Low        |
| 10 | finish coding assignment    | Quiz     | Medium     | 5               | 6.7                | Low        |
| 11 | prepare final report        | Project  | Medium     | 7               | 1.5                | low        |
| 12 | project draft v2            | Quiz     | easy       | 2               | 2                  | low        |
| 13 | hw network lab              | Project  | hard       | 2               | 6.9                | low        |
| 14 | write report draft          | Homewor  | Medium     | 2               | 3.5                | low        |
| 15 | solve math hw ch3           | Project  | Hard       | 4               | 2.8                | Medium     |
| 16 | submit final project report | Homewor  | Easy       | 5               | 2.2                | low        |
| 17 | read article and summarize  | Project  | Easy       | 5               | 3.2                | low        |
| 18 | project draft v7            | Quiz     | Easy       | 1               | 6.4                | high       |
| 19 | Study for 7 midterm!!!      | Quiz     | Hard       | 7               | 3.4                | low        |

Define Dataset Manipulation that will be used in your project with Samples? (**Minimum 1000 words**):

### Dataset Manipulation in the SATM Project

When we first started architecting the Smart Academic Task Manager (SATM), we hit a wall pretty early: students don't talk like machines. They don't provide clean, structured inputs. Instead, they give us fragments of thought, stress-induced shorthand, and a massive amount of digital noise. That's why dataset manipulation became the strategic heart of this project. It wasn't just a chore; it was a battle to turn digital chaos into a structured language that a machine learning model could actually respect.

### The Logic Behind the Pipeline

At its core, manipulation in SATM is all about transformation. We are taking raw, fragmented thoughts and converting them into meaningful numerical vectors. Why put so much effort into this? Because without it, things like task classification and priority scoring are just shots in the dark. We needed a system that could look at a frantic student's notes and find the actual objective priority buried under all that informal slang. It's about building a bridge between how people talk and how computers calculate.

### Cleaning: The Frontline Defense

The "Task Text" column is where the human element is at its loudest—and naturally, where the most noise exists. Our cleaning protocol was designed to strip away the "entropy" of student life. Look at case normalization as an example. It's a tiny tweak, sure, but it's honestly a lifesaver when you're trying to fix vocabulary sparsity. The reality is that a computer sees "FINAL EXAM STUDY 🤯!!!" and "final exam study" as two totally different entities, which just creates messier gradients. By forcing everything into lowercase, we make sure the model doesn't waste its brainpower on capitalization. That frantic, all-caps entry is distilled into a much more manageable "final exam study."

Then, we have to deal with digital artifacts—the emojis and punctuation students use to vent their stress. While a "🤯" emoji tells a human a lot about how a student feels, it's basically junk data for a classification model. We used custom logic to purge these symbols. If an entry looks like "study for phy midterm!!!???", we strip it down to its essence. We don't just "remove" them; we refine the string to "study for phy midterm." It's about getting to the raw signal. We also added whitespace normalization to clear out redundant spaces. This keeps our formatting consistency from getting messy.

Let's be real: students love shorthand. "hw," "ppt," "bio"—this is their natural language. To keep the model consistent, we created a mapping layer to expand these into full terms. We made sure "hw" becomes "homework," "ppt" becomes "presentation," and "bio" becomes "biology." This lexical alignment ensures our AI recognizes a unified academic concept, which is absolutely vital for training. We also had to be pretty strict with low-quality entries. If a student types something vague like "assignment?? idk," our pipeline either cleans it up or tosses it out entirely. We can't have the model learning from junk that lacks a clear intent.

### Translating Thought into Mathematics

Once the text is "human-clean," we have to handle the mathematical translation. Machine learning models are essentially high-level calculators; they don't read words, they process vectors. This is where we bring in Text Vectorization. We chose to use TF-IDF (Term Frequency–Inverse Document Frequency) to weigh word importance. Why? Because in a student's task list, common words like "the" are everywhere but mean nothing, while "calculus" is rare but defines the entire task. TF-IDF helps us highlight these high-value terms so the model knows what to focus on. For more semantic depth, we also looked into Word Embeddings, which allow the model to understand that a "quiz" and a "test" are conceptually cousins even if they are spelled differently.

But SATM isn't just about text. We have categorical metadata like Category, Difficulty, and Importance. Since you can't run an equation on the word "Hard," we used One-Hot Encoding and Label Encoding to map these to numbers. Homework became 0, Exam became 1, Project became 2, and Quiz became 3. On top of that, we applied Min-Max Scaling to numerical features like deadlines and time estimates. We didn't want a 30-day deadline to "outweigh" a 2-hour task estimate just because 30 is a bigger number than 2. Scaling keeps the playing field level and prevents numerical bias from ruining our predictions.

### Engineering the "Smart" Factor

If cleaning is the foundation, Feature Engineering is the architecture. We realized that raw data doesn't always capture the actual feeling of being a student. So, we decided to create our own metrics. The most important one we engineered is the Time Pressure Ratio. Instead of just looking at a date, we compare how long a task takes to how much time is left before the deadline. This gives the model a direct indicator of urgency. We also scan for urgency keywords like "asap" or "urgent" to create binary flags. These features give the model a shortcut to understanding what actually matters right now, going way beyond what's just written in the text.

### Making it all Work

To make sure we weren't just building a system that memorized our own data, we enforced a strict 80/20 data split. We trained the model on 80% of our manipulated data and kept 20% totally hidden—the "dark room" test. This is our primary defense against "overfitting." It's the only way to prove that SATM can handle a brand-new student's input with the same precision as the data we used to build it. It's about ensuring the system actually works in the wild, not just in our lab.

So, how does all this prep actually help? Well, it feeds right into the three main engines of SATM. Take Task Classification—by cleaning the text, we let the model actually see the task type without getting distracted by noise. With Time Estimation, those scaled features are what give our regression models the confidence to predict workload accurately. And for the Priority Scoring, all this structured data is what lets the system blend logic with actual insight.

Honestly, looking back at the whole thing, without scrubbing emojis, mapping shorthands like "hw," and building those pressure ratios, SATM would just be another basic list. Instead, it's a tool that actually understands the student's journey. We realized that refining messy inputs into something useful is the real secret here. It's how we keep students on track in a way that feels natural. And frankly, it makes the whole experience way more helpful for anyone trying to actually survive their semester.

| index | Task Text                    | Category | Difficulty | Deadline (days) | Time Taken (hours) | Importance | Cleaned_Task_Text            | Processed_Task_Text          | Task_Length | Urgency_Flag | Time_Pressure       | Deadline (days)_scaled | Time Taken (hours)_scaled | Difficulty_Encoded | Time Taken (hours)_clipped |
|-------|------------------------------|----------|------------|-----------------|--------------------|------------|------------------------------|------------------------------|-------------|--------------|---------------------|------------------------|---------------------------|--------------------|----------------------------|
| 0     | finish coding assignment     | Exam     | Hard       | 4               | 2.6                | Low        | finish coding assignment     | finish coding assignment     | 3           | 0            | 1.5384615384615383  | -0.03697820230624004   | -0.8433296937333611       | 2                  | 2.6                        |
| 1     | write report draft           | Homework | Medium     | 5               | 4.9                | Low        | write report draft           | write report draft           | 3           | 0            | 1.02040818326953061 | 0.46608219534686677    | 0.5222423635056472        | 1                  | 4.9                        |
| 2     | math problems 1-8            | Project  | Hard       | 7               | 2.8                | High       | math problem 18              | math problem 18              | 3           | 0            | 2.5                 | 1.4782042265019384     | -0.7245845021995171       | 2                  | 2.8                        |
| 3     | project research             | Exam     | Easy       | 7               | 2.4                | Medium     | project research             | project research             | 2           | 0            | 2.8169999999999997  | 1.4782042265019384     | -0.860075466397054        | 0                  | 2.4                        |
| 4     | quiz 1                       | Quiz     | Hard       | 5               | 2.0                | Low        | quiz 1                       | quiz 1                       | 2           | 0            | 2.5                 | 0.46608219534686677    | -1.16059023453348937      | 2                  | 2.0                        |
| 5     | study notes                  | Exam     | Easy       | 3               | 5.1                | High       | study notes                  | study note                   | 2           | 0            | 0.5882352941176471  | -0.5490398358581447    | 0.940987795439991         | 0                  | 5.1                        |
| 6     | finish coding assignment     | Homework | Easy       | 2               | 2.3                | High       | finish coding assignment     | finish coding assignment     | 3           | 0            | 0.8699552173913044  | -1.0521008913896959    | -1.0214481660341278       | 0                  | 2.3                        |
| 7     | project research             | Quiz     | Easy       | 7               | 4.6                | Low        | project research             | project research             | 2           | 0            | 1.5217391304347827  | 1.4782042265019384     | 0.34412423160508054       | 0                  | 4.6                        |
| 8     | finish coding assignment     | Quiz     | Medium     | 5               | 6.7                | Low        | finish coding assignment     | finish coding assignment     | 3           | 0            | 0.7482689567164178  | 0.46608219534686677    | 1.560951157710446         | 1                  | 6.7                        |
| 9     | prepare final report         | Project  | Medium     | 7               | 1.5                | Low        | prepare final report         | prepare final report         | 3           | 0            | 4.669999999999999   | 1.4782042265019384     | -1.4094269821699042       | 1                  | 1.5                        |
| 10    | project draft v2             | Quiz     | Easy       | 2               | 2.0                | Low        | project draft v2             | project draft v2             | 3           | 0            | 1.0                 | -1.0521008913896959    | -1.16059023453348937      | 0                  | 2.0                        |
| 11    | hw network lab               | Project  | Hard       | 2               | 6.9                | Low        | homework network lab         | homework network lab         | 3           | 0            | 0.2988550724637681  | -1.0521008913896959    | 1.709690579244269         | 2                  | 6.9                        |
| 12    | write report draft           | Homework | Medium     | 2               | 3.5                | Low        | write report draft           | write report draft           | 3           | 0            | 0.5714285714285714  | -1.0521008913896959    | -0.3089755988310623       | 1                  | 3.5                        |
| 13    | solve math hw ch3            | Project  | Hard       | 4               | 2.8                | Medium     | solve math homework ch3      | solve math homework ch3      | 4           | 0            | 1.4285714285714286  | -0.03697820230624004   | -0.7245845021995171       | 2                  | 2.8                        |
| 14    | submit final project report  | Homework | Easy       | 5               | 2.2                | Low        | submit final project report  | submit final project report  | 4           | 0            | 2.2727272727272725  | 0.46608219534686677    | -1.0806208268010466       | 0                  | 2.2                        |
| 15    | read article and summarise   | Project  | Easy       | 5               | 3.2                | Low        | read article and summarise   | read article summarise       | 3           | 0            | 1.5625              | 0.46608219534686677    | -0.4870987181318236       | 0                  | 3.2                        |
| 16    | project draft v7             | Quiz     | Easy       | 1               | 6.4                | High       | project draft v7             | project draft v7             | 3           | 0            | 0.15025             | -1.5581618969631863    | 1.4128330254066789        | 0                  | 6.4                        |
| 17    | Study for 7 midterm!!!       | Quiz     | Hard       | 7               | 3.4                | Low        | study for 7 midterm          | study 7 midterm              | 3           | 0            | 2.05823529411765    | 1.4782042265019384     | -0.3683482975978443       | 2                  | 3.4                        |
| 18    | study notes                  | Exam     | Hard       | 7               | 4.0                | High       | study notes                  | study note                   | 2           | 0            | 1.75                | 1.4782042265019384     | -0.012102102596461827     | 2                  | 4.0                        |
| 19    | weekly 10 quiz??             | Project  | Easy       | 6               | 4.4                | High       | weekly 10 quiz               | weekly 10 quiz               | 3           | 0            | 1.3636363636363635  | 0.8721432109244175     | -0.22373781097123672      | 0                  | 4.4                        |
| 20    | chem quiz tomorrow           | Project  | Easy       | 2               | 2.9                | Medium     | chem quiz tomorrow           | chem quiz tomorrow           | 3           | 1            | 0.690551724137931   | -1.0521008913896959    | -0.8652118514329949       | 0                  | 2.9                        |
| 21    | solve math hw ch2            | Exam     | Medium     | 2               | 2.2                | Medium     | solve math homework ch2      | solve math homework ch2      | 4           | 0            | 0.9069069069069091  | -1.0521008913896959    | -1.0806208268010466       | 1                  | 2.2                        |
| 22    | study slides quickly         | Project  | Easy       | 5               | 1.5                | High       | study slide quickly          | study slide quickly          | 3           | 0            | 3.3333333333333335  | 0.46608219534686677    | -1.4864298021699042       | 0                  | 1.5                        |
| 23    | Group project meeting        | Exam     | Easy       | 4               | 1.1                | Medium     | group project meeting        | group project meeting        | 3           | 0            | 3.9363636363636362  | -0.03697820230624004   | -1.7336205452371826       | 0                  | 1.1                        |
| 24    | big project final submission | Homework | Medium     | 1               | 5.0                | High       | big project final submission | big project final submission | 4           | 0            | 0.2                 | -1.5581618969631863    | 0.5816150740727041        | 1                  | 5.0                        |

Show 25 per page

1 2 10 30 40

### Overview of the proposed application system with screenshots? (Minimum 500 words):

#### System Overview

After building the AI components in the earlier sections, we needed a way for students to actually use them. That is where the Smart Academic Task Manager (SATM) comes in. It is a web-based application that ties everything together — the NLP module, the classification model, the priority scoring formula, and the dataset-driven logic — and puts them behind a simple interface a student can open in a browser. We did not want it to feel like a regular to-do

list, because it is not one. The to-do list part is just the surface; underneath, the AI components are doing most of the work.

The way we set it up, almost every action the student takes ends up triggering one of the AI parts in the background. Typing in a task description sends the text through the NLP module. Saving a task runs it through the classification model. Opening the dashboard pulls in the latest priority scores. The student does not see any of this happening directly, which is the point — the AI should feel like a helpful layer, not a feature the user has to think about.

The application is split into three main screens: the Dashboard, the Add Task screen, and the Settings screen. Each one handles a different stage of the workflow. Together, they cover the full path from a student typing a rough task description to seeing a clean, ranked list of what to work on next.

#### Screen 1 — Dashboard

The Dashboard is what the student lands on when they open the app. The idea behind it is simple: before asking the student to do anything, give them a clear picture of where they currently stand academically. No typing, no clicking through menus — just a quick summary at the top of the page.

Four numbers are shown across the top: how many active tasks the student has, how many of those are due today, how many have been completed, and how accurate the AI auto-fill has been so far. These are not hardcoded; they update in real time based on the task data stored in the system, so whenever a task is added or marked done, the numbers shift accordingly.

Right under that summary is the upcoming task list, which is probably the most useful part of the screen. The tasks here are not sorted by date — they are ranked by Priority Score, the same score produced by the hybrid scoring model from the AI section. Each row shows the task name, its category, the due date, and an importance badge, so the student can glance at the list and immediately see what the model thinks is the most urgent.

Further down, there is a course progress section and an AI Suggestion panel. The Suggestion panel is where the scheduling recommendation part of the pipeline shows up visually. It picks one task it thinks the student should focus on first, gives a rough estimate of how many hours of work are still left, and adds a conflict warning if two high-priority tasks happen to fall in the same deadline window. It is meant to feel less like a notification and more like a quick piece of advice from the system.

#### Screen 2 — Add Task: Turning Text into Action

This screen isn't just for typing; it's where the actual "heavy lifting" of the AI happens. We wanted to make sure a student could just throw in a task description—using normal, everyday language—and let the system do the rest. Once you hit enter, our NLP pipeline kicks in, cleaning up the text and pulling out the keywords that actually matter.

From there, the app automatically fills in the **Category** (like a quiz or a project), the **Deadline**, and how **Important** it is. This logic isn't guesswork; it follows the same classification rules we used to train our dataset.

**One thing we really focused on:** Keep the student in control. After the system calculates a priority score (shown as a color-coded bar), we provide a suggestion, but it's not final. Every

field is an editable dropdown. We call this a "**human-in-the-loop**" design—the AI suggests the details, but you have the final word before it ever touches your queue.

### Screen 3 — Settings: User Adjustments

Screen 3 is basically the dashboard for how the AI acts. We added a few toggles in the "AI Behaviour" section so users can decide for themselves if they want the auto-fill on or if they need the system to flag "conflicting deadlines." It's all about giving you control over how much the app intervenes in your planning.

You can also change the theme or switch the language. Switching to Arabic is actually a pretty big technical deal—it's not just a translation. It forces the NLP module to swap out its entire tokenization and word-removal logic to handle the unique structure of Arabic grammar, which is a key part of our long-term plan for the app.

### The Full Loop

Basically, these screens function as one connected system. You feed raw ideas into **Screen 2**, the Dashboard visualizes the results, and **Screen 3** is where you go to tweak the settings until the app works exactly how you want. Everything you see—every button or toggle—is just a bridge to the AI pipeline working in the background.



Link? (**If it is available**):

[SATM — Smart Academic Task Manager](#)